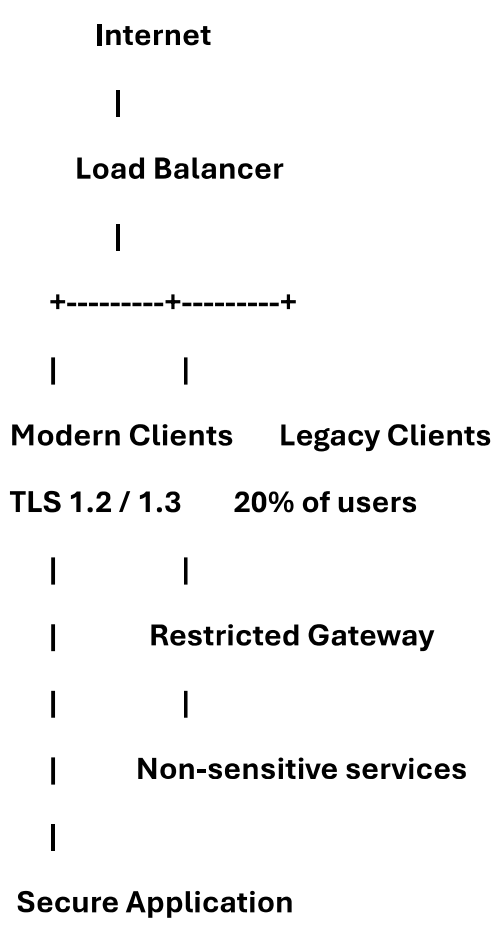
1)Solution: Secure TLS Migration for a Financial Institution

The financial institution should use a phased TLS migration strategy in which modern clients use TLS 1.2/1.3, while legacy clients are isolated and gradually upgraded. Since sensitive information must not travel over SSL, legacy clients should not be allowed to access sensitive transactions directly.

Proposed Architecture



A. Maintaining Backward Compatibility

- **Support TLS 1.2 and TLS 1.3 as the standard protocols.**
- **Keep a separate legacy gateway for the 20% of clients that cannot immediately support modern TLS.**
- **Do not enable SSL for sensitive transactions.**

- **Restrict legacy clients to non-sensitive functions such as:**
    - **Public information**
    - **Account-support information**
    - **General notifications**
- **Display an upgrade requirement when a legacy client attempts to perform a sensitive operation.**

This maintains limited backward compatibility without exposing financial data through obsolete SSL protocols.

B. Protecting Sensitive Data

Sensitive information such as:

- **Account numbers**
- **Passwords**
- **Credit/debit-card information**
- **Transaction details**
- **Personal financial information**

must only be transmitted through TLS 1.2 or preferably TLS 1.3.

The server should disable SSL 2.0 and SSL 3.0 and avoid weak TLS configurations such as obsolete cipher suites.

Additional protections should include:

- **Strong certificate management**
- **Secure cipher suites**
- **HSTS where appropriate**
- **Secure cookies**
- **Certificate monitoring and timely renewal**
- **Strong authentication such as MFA for financial operations**

C. Minimizing Downtime

Use a phased migration rather than replacing the entire system at once.

Phase 1 – Assessment

- **Identify all legacy clients.**
- **Determine exactly which systems require older protocols.**
- **Monitor current TLS/SSL traffic.**

Phase 2 – Dual operation

- **Modern clients use TLS 1.2/1.3.**
- **Legacy clients are temporarily routed through the isolated legacy gateway.**
- **Monitor errors and compatibility problems.**

Phase 3 – Client upgrade

- **Upgrade legacy applications and operating systems gradually.**
- **Provide users with updated software or supported browsers.**
- **Test each group before migration.**

Phase 4 – SSL elimination

- **Disable SSL completely once the legacy population has been migrated.**
- **Remove the legacy gateway after confirming that no critical users depend on it.**

Using load balancers and rolling deployments allows servers to be upgraded without taking the entire service offline.

D. Optimizing Implementation Cost

The institution can reduce costs by:

1. **Using existing load balancers/reverse proxies to terminate TLS.**
2. **Centralizing certificate management.**
3. **Prioritizing upgrades for systems handling sensitive transactions.**
4. **Using open standards and existing TLS-capable infrastructure rather than replacing every system immediately.**
5. **Reusing existing servers where their software can be upgraded securely.**
6. **Automating certificate renewal and configuration deployment.**

Recommended Security Policy

Client Type	Protocol	Sensitive Transactions	Action
Modern clients	TLS 1.3	✅ Allowed	Preferred
Compatible clients	TLS 1.2	✅ Allowed	Supported
Legacy clients	Older SSL	❌ Not allowed	Restrict/upgrade
Unsupported clients	SSL/weak protocols	❌ Not allowed	Block

Final Recommendation

The institution should adopt a TLS 1.2/1.3-first architecture with an isolated legacy gateway. The legacy gateway should provide only limited, non-sensitive functionality and should never carry passwords, financial transactions, or other sensitive information over SSL.

This approach provides the best balance of the four constraints:

- **Backward compatibility: 20% legacy users can temporarily access limited services.**
- **Security: Sensitive transactions are exclusively protected by modern TLS.**
- **Minimal downtime: Phased migration and rolling deployment avoid major outages.**
- **Cost optimization: Existing infrastructure can be reused while legacy systems are upgraded gradually.**

Long-term goal: eliminate SSL completely and standardize the institution on TLS 1.3, with TLS 1.2 retained only where necessary for compatibility.

2) Decision: SET vs TLS for E-Commerce Payment Security

The e-commerce platform should continue using TLS-based payment security rather than implementing SET (Secure Electronic Transaction).

SET was designed specifically for card-payment security, but it is complex and is not the practical choice for a modern, high-throughput e-commerce platform. TLS provides strong protection for communication and is widely supported by modern browsers.

Comparison

Factor	SET	TLS
Transaction throughput	More processing and protocol complexity	High throughput with modern TLS implementations
Modern browser compatibility	❌ Poor / obsolete	✅ Excellent
Implementation complexity	❌ High	✅ Relatively low
Payment security	Strong, designed specifically for cards	Strong transport encryption
PCI-DSS suitability	Can support compliance	Widely used in compliant payment architectures
Deployment	Complex certificates and infrastructure	Easier with existing web infrastructure
Scalability	More difficult	Highly scalable

A. High Transaction Throughput

The system must support 10,000 transactions/minute, which is approximately:

10,000 ÷ 60 = 166.7 transactions/second

TLS is more suitable because modern TLS implementations support session reuse, efficient cryptographic algorithms, and hardware acceleration.

A scalable architecture can use:

Customers

↓

Load Balancer

↓

TLS Termination

↓

Web/Application Servers

↓

Payment Gateway

↓

Bank/Card Network

This architecture can distribute thousands of transactions across multiple application servers.

B. Modern Browser Compatibility

TLS should be selected.

Modern browsers natively support TLS, particularly TLS 1.2 and TLS 1.3. SET, on the other hand, is an outdated payment protocol and is not supported as a normal browser-based payment mechanism.

Therefore, adopting SET would introduce unnecessary compatibility problems.

C. Minimizing Implementation Complexity

TLS is significantly simpler because the platform likely already has:

- **HTTPS support**
- **Digital certificates**
- **TLS-capable web servers**
- **Reverse proxies/load balancers**
- **Existing browser compatibility**

SET would require additional infrastructure for its certificate-based payment protocol and specialized transaction processing.

Therefore, TLS has substantially lower implementation and maintenance complexity.

D. PCI-DSS Compliance

TLS can contribute to PCI-DSS compliance by protecting payment-card data during transmission. However, using TLS alone does not automatically make the platform PCI-DSS compliant.

The organization should also implement appropriate controls such as:

- **Use strong, currently supported TLS configurations.**
- **Disable obsolete protocols and weak cryptographic algorithms.**
- **Minimize storage of cardholder data.**
- **Use a PCI-compliant payment gateway where possible.**
- **Apply access controls and least privilege.**
- **Maintain logging and monitoring.**
- **Regularly perform vulnerability assessments and security testing.**
- **Protect stored cardholder data where storage is necessary.**

Recommended Decision

Do not implement SET. Continue with TLS, preferably TLS 1.3 with TLS 1.2 where compatibility requires it.

For payment processing, an even better architecture is:

Customer Browser

↓

HTTPS/TLS

↓

E-Commerce Platform

↓

PCI-Compliant Payment Gateway

↓

Bank/Card Network

This approach provides:

- ✅ **10,000+ transactions/minute scalability**
- ✅ **Modern browser compatibility**
- ✅ **Lower implementation complexity**
- ✅ **Strong encryption in transit**
- ✅ **Easier integration with existing infrastructure**
- ✅ **Support for PCI-DSS compliance when combined with the required organizational and technical controls**

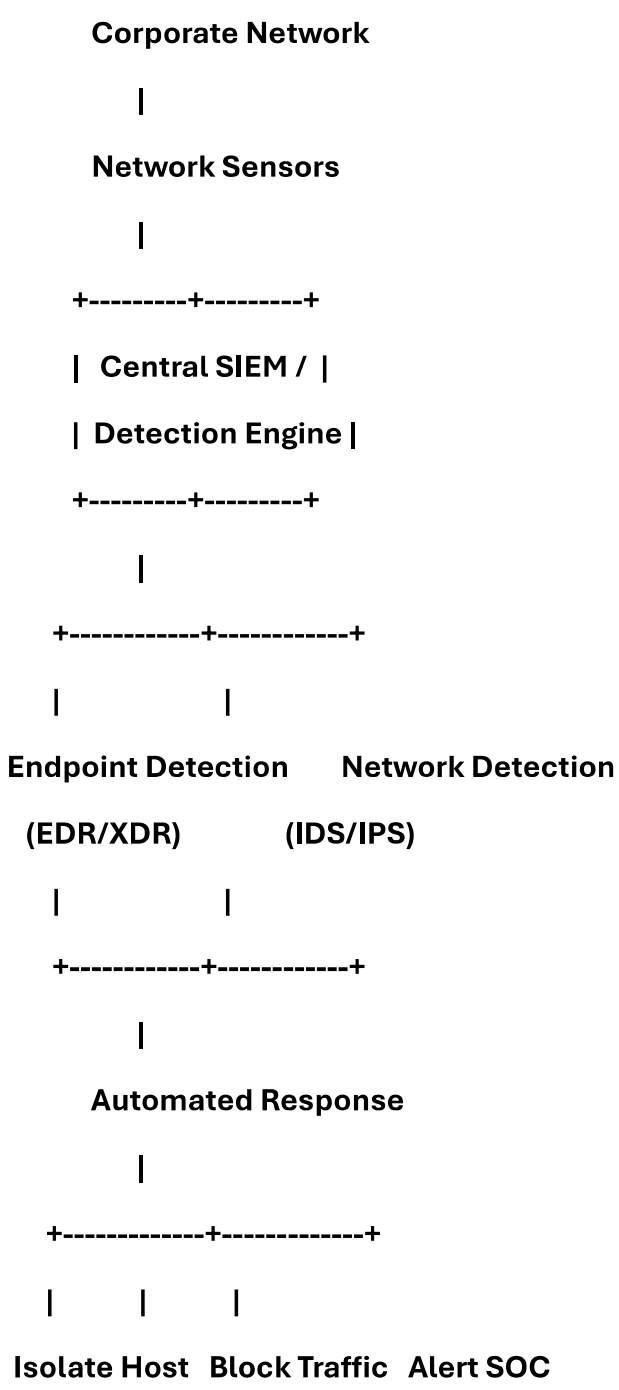
Final Conclusion

TLS is the clear choice over SET for this e-commerce platform. SET's specialized payment-security features do not justify its complexity and lack of modern browser support. A TLS 1.3-based architecture integrated with a PCI-compliant payment gateway provides the best balance of performance, compatibility, security, and operational simplicity.

3) Malware Containment and Prevention Strategy

The symptoms suggest worm-like behavior, because the malware is spreading rapidly across interconnected systems without requiring user interaction. However, the solution should classify the infection as a virus, worm, or Trojan before applying the appropriate response.

Proposed Architecture



1. Detection Within 2 Minutes

Deploy EDR/XDR agents across all 500+ systems and integrate them with network IDS/IPS.

Monitor for:

- **Sudden abnormal network connections**
- **Rapid connection attempts to multiple systems**
- **Unexpected file modifications**
- **Unauthorized process execution**
- **Registry/system configuration changes**
- **Suspicious PowerShell or scripting activity**
- **Unusual CPU/network activity**
- **Attempts to exploit network services**

Use automated correlation in a SIEM so that suspicious events are detected and classified immediately.

Target: Detection and automated alerting within 2 minutes.

2. Differentiate Virus, Worm, and Trojan

Malware	Typical Behavior	Detection Indicators	Response
Virus	Attaches to legitimate files and spreads when infected files are executed	File modification, executable infection, unusual file hashes	Quarantine infected files and restore clean versions
Worm	Self-propagates across networks without user interaction	Rapid scanning, repeated connections, exploitation attempts	Immediately isolate infected host and block propagation traffic
Trojan	Disguises itself as legitimate software and generally requires execution by a user/process	Suspicious executable, unexpected outbound connections, persistence mechanisms	Kill process, quarantine software, investigate credentials

The rapid, user-independent propagation in the scenario should trigger the highest-priority worm containment policy.

3. Automated Containment

Because there are more than 500 interconnected systems, manual isolation would be too slow.

When EDR detects a high-confidence infection:

1. **Automatically isolate the affected endpoint using network access control/EDR.**
2. **Block its suspicious outbound connections.**
3. **Prevent communication with vulnerable systems.**
4. **Alert the security operations team.**
5. **Search other endpoints for the same indicators of compromise.**
6. **Quarantine additional infected machines automatically.**
7. **Preserve forensic information for investigation.**

Only the infected system should initially be isolated rather than shutting down entire network segments.

4. Keep Downtime Below 10 Minutes

To satisfy the downtime constraint:

- **Use automated endpoint isolation instead of manual shutdown.**
- **Maintain endpoint images/backups for rapid recovery.**
- **Keep security agents and operating systems patched.**
- **Use redundant network and security infrastructure.**
- **Automatically restore or reimage compromised endpoints where appropriate.**
- **Test recovery procedures regularly.**

A practical target is:

Detection → Isolation → Cleanup/Recovery: ≤ 10 minutes per system

5. Prevention Strategy

Network controls

- **Segment the network using VLANs/firewalls.**
- **Restrict unnecessary host-to-host communication.**
- **Apply network access control (NAC).**
- **Use IDS/IPS to detect exploit and scanning behavior.**
- **Block known malicious domains and IP addresses.**

Endpoint controls

- **Deploy EDR/XDR on all systems.**
- **Keep operating systems and applications patched.**
- **Use application allowlisting where practical.**
- **Disable unnecessary services and protocols.**
- **Restrict administrator privileges.**

Email/Web controls

- **Scan attachments and URLs.**
- **Block executable attachments where appropriate.**
- **Use email authentication and anti-malware filtering.**
- **Apply web/DNS filtering.**

6. Response Workflow

Infection

↓

EDR/IDS Detection (< 2 min)

↓

Behavior Classification

↓

Virus / Worm / Trojan

↓

Automated Isolation

↓

Block IOCs + Stop Propagation

↓

Scan Other 500+ Systems

↓

Remove Malware / Patch Vulnerability

↓

Restore System

↓

Monitor for Reinfection

Recommended Strategy

The best solution is a centralized EDR/XDR + SIEM + IDS/IPS + network segmentation + automated containment system.

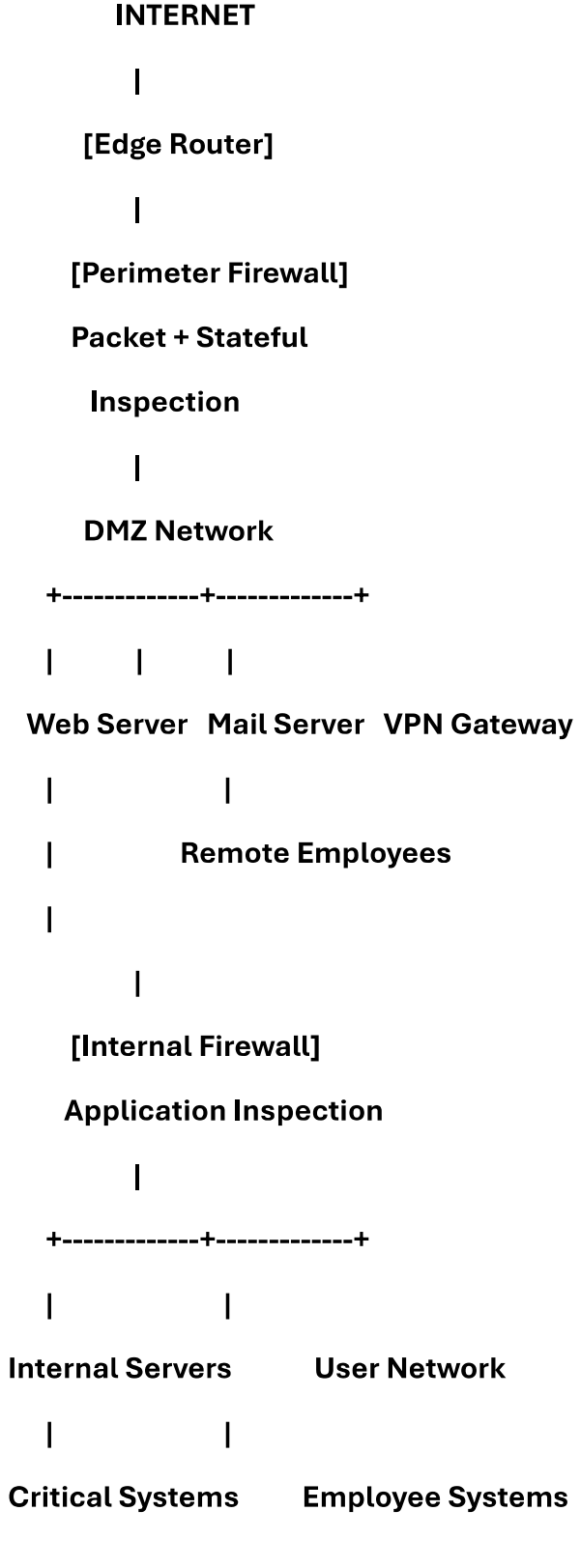
For this scenario, worm behavior should receive the highest-priority automated response, because a worm can spread independently across hundreds of interconnected systems. Automated detection and isolation can meet the 2-minute detection requirement, while rapid reimaging and recovery can keep individual system downtime within the 10-minute limit.

Final recommendation: Use behavior-based detection rather than relying only on malware signatures. Combining endpoint and network telemetry allows the organization to distinguish virus, worm, and Trojan behavior and apply the appropriate containment response without unnecessarily taking the entire 500+ system network offline.

4) Firewall Architecture Design

The organization should deploy a hybrid, multi-layer firewall architecture combining packet filtering, stateful inspection, and application-level inspection, along with a VPN gateway for secure remote employees.

Proposed Architecture



1. Packet Filtering

The perimeter firewall should perform packet filtering and stateful inspection.

It should examine:

- **Source and destination IP addresses**
- **Source and destination ports**
- **Protocol (TCP/UDP/ICMP)**
- **Connection state**
- **Interface and network zone**

Example policies:

Traffic	Action
Internet → Internal network	Deny by default
Internet → Public web server	Allow required HTTPS
Internet → Internal database	Deny
Internal → Internet	Allow according to policy
DMZ → Internal network	Strictly restricted
Remote VPN → Internal network	Allow only authorized resources

A default-deny policy should be used, with only required traffic explicitly allowed.

2. Application-Level Inspection

A next-generation firewall (NGFW) or application-aware firewall should inspect traffic at the application layer.

It can identify and control:

- **HTTP/HTTPS applications**
- **DNS traffic**
- **Email protocols**
- **File transfers**
- **Unauthorized applications**
- **Malicious payloads**
- **Suspicious application behavior**

For encrypted HTTPS traffic, TLS inspection can be used where legally and operationally appropriate, with careful exclusions for privacy-sensitive traffic.

3. Protection Against Insider Attacks

A single perimeter firewall is not sufficient against insider threats.

Therefore, deploy internal segmentation/firewalls between important zones:

User Network

|

| Internal Firewall

↓

Application Servers

|

| Internal Firewall

↓

Database / Critical Systems

This limits lateral movement if an employee workstation becomes compromised.

Additional controls should include:

- **Network segmentation**
- **Role-based access control**
- **IDS/IPS**
- **Endpoint Detection and Response (EDR)**
- **Security logging and SIEM monitoring**
- **Least-privilege access**

4. Latency Requirement — ≤ 5 ms

To keep firewall latency below 5 ms per request:

- **Use dedicated firewall appliances or high-performance virtual appliances.**
- **Enable hardware acceleration where available.**
- **Avoid unnecessary deep-packet inspection on every connection.**
- **Use efficient firewall rules.**
- **Keep frequently used rules near the top of the policy.**
- **Use load balancing/high availability for heavily used services.**
- **Monitor latency continuously.**

Application-level inspection should be selectively applied to high-risk or relevant traffic rather than unnecessarily processing every packet with expensive inspection.

5. Traffic Requirement — 1 Gbps

The firewall should be rated for more than 1 Gbps of real-world inspected throughput, rather than simply having a nominal 1-Gbps interface.

A suitable design would provide additional capacity, for example:

Required traffic: 1 Gbps
Recommended firewall capacity: ≥ 2 Gbps inspected throughput

This provides headroom for traffic growth and security inspection overhead.

Use active/standby or active/active firewall redundancy so that failure of one firewall does not interrupt the entire network.

6. Secure Remote Access

Employees should connect through an IPSec VPN or SSL/TLS-based VPN terminating at the VPN gateway.

Remote Employee

|

Encrypted VPN

|

VPN Gateway

|

Authentication + MFA

|

Internal Firewall

|

Authorized Resources

Security measures should include:

- **Multi-factor authentication (MFA)**
- **Strong encryption**
- **Certificate or strong credential-based authentication**
- **Device security checks where supported**
- **Role-based access**
- **Session logging**
- **Automatic timeout**
- **Access only to required internal resources**

Final Recommended Architecture

Requirement	Solution
Packet filtering	Stateful perimeter firewall
Application inspection	NGFW/application-aware firewall

Requirement	Solution
Insider protection	Internal segmentation firewall
≤ 5 ms latency	High-performance hardware + optimized policies
≥ 1 Gbps traffic	Firewall rated above 1 Gbps inspected throughput
Remote access	IPSec/SSL VPN + MFA
External protection	Perimeter firewall + IDS/IPS
Internal protection	Segmentation + internal firewall
Availability	High-availability firewall pair
Monitoring	Centralized SIEM/log management

Conclusion

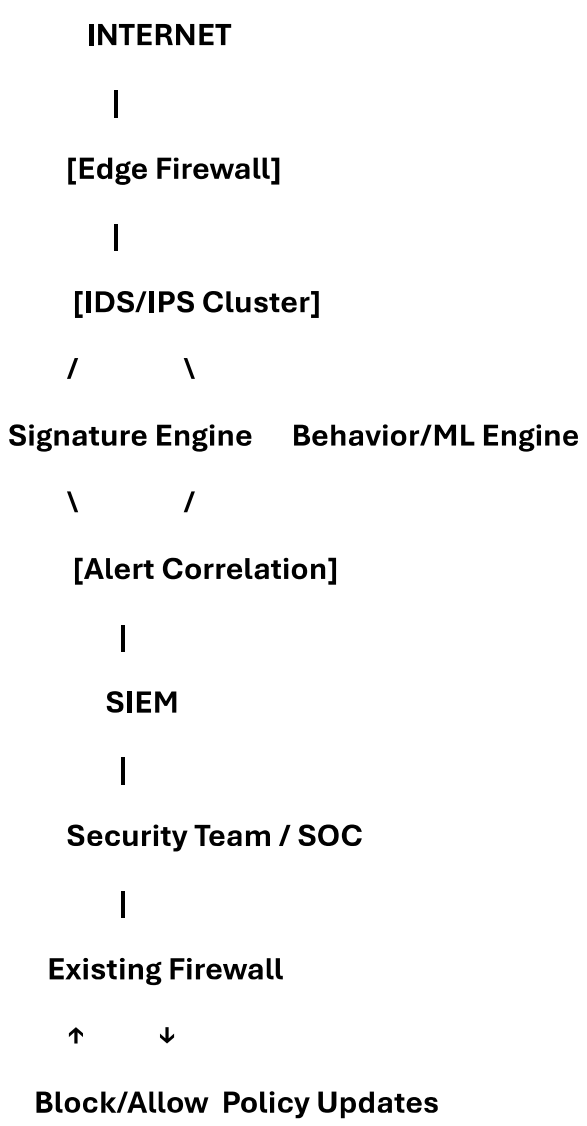
The most suitable solution is a multi-layer firewall architecture consisting of a high-performance perimeter NGFW, DMZ, internal segmentation firewall, IDS/IPS, and secure VPN gateway.

This design provides packet filtering and application-level inspection, protects against both external and insider threats, supports at least 1 Gbps traffic, and provides secure remote access. By using high-performance hardware, optimized inspection policies, and selective deep inspection, the architecture can be designed to keep firewall-induced latency within the 5 ms requirement.

5) Optimized IDS/IPS Solution for a Large Enterprise

The organization should deploy a hybrid, high-performance IDS/IPS architecture combining signature-based detection, machine-learning/behavior-based detection, traffic baselining, and centralized alert correlation. The main goal is to reduce false positives without sacrificing detection accuracy.

Proposed Architecture



1. Improve Detection Accuracy to ≥95%

Use a multi-layer detection approach:

- **Signature-based detection → identifies known attacks.**
- **Behavior-based detection → identifies abnormal traffic patterns.**
- **Anomaly detection/ML → detects previously unknown threats.**
- **Threat-intelligence feeds → identify known malicious IPs, domains, and hashes.**
- **Protocol-aware inspection → understands HTTP, DNS, SMTP, TCP, etc.**

Instead of relying on a single detection method, correlate multiple indicators before generating a high-priority alert.

2. Reduce False Positives Below 5%

Alert fatigue is mainly caused by treating every suspicious event as a critical alert. Use risk-based alerting.

For example:

Event	Risk	Action
Known malicious IP + suspicious payload	High	Block + alert
Multiple failed connections	Medium	Monitor/correlate
Unusual but legitimate application traffic	Low	Log
Confirmed exploit signature	Critical	Block immediately

Additional techniques:

- **Establish a normal traffic baseline for each network segment.**
- **Whitelist verified business applications and trusted services.**
- **Tune signatures to the organization's environment.**
- **Remove obsolete or duplicate rules.**
- **Correlate multiple events before generating an alert.**
- **Continuously retrain anomaly-detection models using validated events.**
- **Use feedback from security analysts to improve detection rules.**

The objective is to achieve:

False Positive Rate < 5%

without weakening detection of genuine attacks.

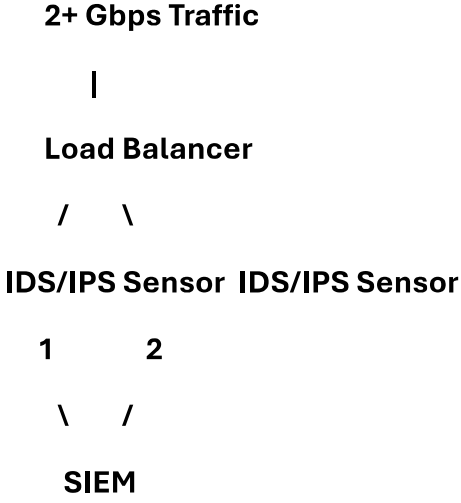
3. Support 2 Gbps Real-Time Traffic

The IDS/IPS should be sized for more than 2 Gbps of sustained inspected traffic, rather than exactly 2 Gbps.

A suitable design would use:

- **High-performance IDS/IPS appliances**
- **Multi-core packet processing**
- **Hardware acceleration where available**
- **Load balancing across multiple sensors**
- **Network taps or high-performance traffic mirroring**
- **Selective deep-packet inspection**
- **High-speed interfaces**

For example:



This allows traffic inspection to scale horizontally and prevents a single sensor from becoming a bottleneck.

4. Integration with Existing Firewall

The IDS/IPS should be integrated with the existing firewall through automated threat-response mechanisms.

Internet

↓

Existing Firewall

↓

IDS/IPS

↓

Internal Network

↓

SIEM/SOC

↓

Firewall Policy Update

When the IDS detects a confirmed attack, it can send an automated response to the firewall to:

- **Block malicious IP addresses**
- **Block malicious domains**
- **Drop suspicious connections**
- **Isolate affected network segments**
- **Apply temporary access-control rules**

However, automatic blocking should generally be limited to high-confidence detections to avoid legitimate traffic being accidentally blocked.

5. Centralized SIEM and Alert Correlation

All IDS/IPS events should be forwarded to a SIEM.

The SIEM can correlate:

- **IDS alerts**
- **Firewall logs**
- **Authentication events**
- **Endpoint security events**
- **DNS activity**
- **Network flow information**

Instead of generating ten separate alerts for the same attack, correlation can produce one prioritized incident.

Recommended Configuration

Requirement	Proposed Solution
Detection accuracy ≥95%	Hybrid signature + behavior/ML detection
False positives <5%	Baselines + rule tuning + correlation + risk scoring
2 Gbps real-time traffic	High-performance clustered IDS/IPS
Firewall integration	Automated firewall policy updates
Alert fatigue	SIEM correlation and risk-based prioritization
Unknown attacks	Anomaly/behavior detection
Known attacks	Signature + threat intelligence
Availability	Redundant IDS/IPS sensors

Final Recommendation

The best solution is a hybrid IDS/IPS cluster integrated with the existing firewall and SIEM. Signature detection should handle known attacks, while behavior-based and ML detection should identify evolving or previously unknown threats.

The system should use traffic baselining, whitelist tuning, risk-based scoring, and event correlation to bring the false-positive rate below 5%. A clustered architecture rated above 2 Gbps inspected throughput ensures real-time processing without creating a network bottleneck.

This approach provides the required balance of ≥95% detection accuracy, <5% false positives, 2 Gbps performance, and seamless firewall integration.